UNIVERSIDADE FEDERAL DE VIÇOSA

CAMPUS FLORESTAL

Guia de Padrões de Testes

Projeto: Pingo - Uma Aventura na Rede

Florestal - MG

2025

Sumário

1. Objetivo.....3
2. Estrutura Padronizada de Testes.....3
2.1. Testes Unitários.....3
2.2. Testes de Integração.....3
2.3. Testes de API/Controller.....4
3. Regras e Convenções para Mocks.....4
4. Regras e Convenções para Cenários de Teste.....4
5. Estrutura de Pastas.....5
6. Execução no Pipeline.....5
7. Checklist para Revisão de Testes.....5

1. Objetivo

Este documento define as convenções e boas práticas para a criação de testes automatizados no backend do projeto. O objetivo é garantir qualidade, consistência e manutenibilidade, permitindo que todos os desenvolvedores sigam a mesma estrutura de testes definida pela arquitetura.

2. Estrutura Padronizada de Testes

Cada tipo de teste deve seguir uma estrutura pré-definida, alinhada à arquitetura em camadas (Controller, Service, DAO, Model).

2.1. Testes Unitários

- Escopo: testar classes isoladas (ex.: métodos de `Service`).
- Ferramentas: **JUnit 5 + Mockito**.
- Estrutura: **Arrange → Act → Assert**.
- Dependências externas devem ser simuladas com mocks.

Exemplo:

when(userDao.findById(1)).thenReturn(user);

2.2. Testes de Integração

- Escopo: validar a interação entre camadas (**Service ↔ DAO ↔ Banco**).
- Usar **H2 em memória** para simular o MySQL.
- Pode-se popular dados de teste com **@Sql** quando necessário.

Exemplo:

@Sql("/insert_test_data.sql")

2.3. Testes de API/Controller

- Escopo: validar endpoints REST.
- Ferramenta: **Spring MockMvc**.
- Validar **status code** e **resposta JSON**.

Exemplo:

mockMvc.perform(get("/api/users")).andExpect(status().isOk());

3. Regras e Convenções para Mocks

- Usar mocks apenas quando a dependência não for o foco do teste.
    - Exemplo: mockar o DAO em testes de Service.
- Ferramentas recomendadas: **Mockito** (para Java).
- Convenções:
    - Nomear mocks com sufixo **Mock**.
        - Exemplo: **userDaoMock**.
    - Configurar comportamento esperado com **when(...).thenReturn(...)**.
    - Validar interações com **verify(...)** sempre que necessário.

4. Regras e Convenções para Cenários de Teste

- Cada teste deve cobrir **um único cenário** (caso de uso).

Nome dos métodos de teste deve ser descritivo, no formato:

metodoASerTestado_condicaoEsperada_resultadoEsperado

- Exemplo:

    - **getUserById_whenIdIsValid_shouldReturnUser()**
    - **getUserById_whenUserNotFound_shouldThrowException()**
- Para cada funcionalidade, devem ser definidos ao menos:
    - **Cenário positivo**: comportamento esperado em condições normais.
    - **Cenário negativo**: entradas inválidas ou erros esperados.
    - **Cenário de borda**: valores limite (ex: id = 0, string vazia).

5. Estrutura de Pastas

Os pacotes de teste devem espelhar **exatamente** a estrutura de pacotes definida no documento de Arquitetura da Aplicação - V1.

Exemplo:

src/test/java/com/pingo/controller

src/test/java/com/pingo/dao

src/test/java/com/pingo/model

src/test/java/com/pingo/service

6. Execução no Pipeline

- Todos os testes devem rodar no **CI/CD (GitHub Actions)** antes de mergear um PR.
- Branches só podem ser aprovadas se a suíte de testes passar.
- O banco em memória (H2) deve ser usado para garantir independência do ambiente.

7. Checklist para Revisão de Testes

Antes de aprovar um PR com testes, verificar:

- Segue o padrão Arrange → Act → Assert?
- Tem cenários positivos, negativos e de borda?
- Mocks foram usados corretamente?
- Os nomes dos métodos são claros e consistentes?

- Estrutura de pacotes espelha o código fonte?
- Testes rodam no pipeline sem dependência externa?